

ECSE 429 Term Project: Part B

Story Testing of REST API

Ethan Wu, 261117309
Kenny Duy Nguyen, 261120429
Emile Labrunie, 261097953
Yanzhe Zhang, 261016377

March 17th, 2026

Version 1

Contents

1	Summary of Deliverables	2
2	Structure of the Story Test Suite	2
2.1	Gherkin feature files	2
2.2	Acceptance test flows	3
3	Source Code Repository	4
3.1	Step definition library	4
3.2	Key files and directory layout	4
3.3	System state and independence	4
3.4	Running the tests and server dependency	5
4	Findings of Story Test Suite Execution	5
4.1	Test execution summary	5
4.2	Bug summary	5
5	Conclusion	7

1 Summary of Deliverables

This report details the design, implementation, and execution of a behaviour-driven development (BDD) story test suite for the Todo Manager REST API. The objective of this phase was to translate system requirements into executable user stories using Gherkin syntax and automate their verification using Cucumber.

Throughout the project, we interpreted the API in a real-world context: **priorities** are represented as category objects, **course todo lists** as project objects, and **tasks** as todo objects. This mapping allowed the user stories to validate the API in a consistent, scenario-based manner. The remainder of this report is organised as follows: Section 2 describes the structure of the story test suite; Section 3 describes the source code repository and how to run the tests; Section 4 summarises the findings of test execution and the bugs discovered; and Section 5 concludes.

Our deliverables for Part B include:

- **Feature files:** A comprehensive suite of 20 user stories (5 per team member), defined using Gherkin Scenario Outlines. Each story includes at least a Normal flow, an Alternate flow, and an Error flow. Each feature file uses a **Background** section to set initial conditions.
- **Step definitions library:** A reusable, modular JavaScript library utilizing Axios to execute API calls and assert system states. Shared steps are centralised in `sharedSteps.js` and `api.js`; story-specific steps follow the naming convention `StoryNNameSteps.js`.
- **Automated test suite:** A fully automated Cucumber.js environment that ensures tests are independent, order-agnostic, and capable of restoring the system to its initial state via **Before** and **After** hooks.
- **Video demonstration:** A recording showcasing the execution of the test suite in multiple orders and demonstrating the fail-safe behaviour when the API service is offline.
- **Bug summaries:** Documentation of defects discovered during automated story execution (see Section 4).
- **This written report,** including sections that summarize deliverables, describe the structure of the acceptance test suite, describe the source code repository, and summarize the findings of story test suite execution, in line with the Part B rubric.

2 Structure of the Story Test Suite

The test suite is built on BDD principles to ensure traceability between user requirements and API behaviour. Each team member defined 5 distinct user stories using the standard format: *As a [role], I want [action] so that [value]*.

2.1 Gherkin feature files

The user stories are stored as `.feature` files under `partB/story_test_suite/features/`. There are 20 feature files, each named by the convention `StoryNStoryName.feature` (e.g. `Story1CreateTodo.feature`, `Story3UpdateTodoDescription.feature`).

To maximise test coverage and ensure deterministic execution, each feature file incorporates:

- **Background:** Used to establish the initial system state (e.g. ensuring the server is running, seeding baseline categories, projects, or TODOs) prior to executing scenarios.

- **Scenario outlines:** Utilised to run data-driven tests. By using **Examples** tables, we execute the same behavioural logic against multiple sets of variable data.

Table 1 lists all 20 stories and the flows implemented for each. Every story has at least one Normal, one Alternate, and one Error flow, as required by the rubric.

Story	Feature	Normal	Alternate	Error
1	Create Todo	✓	✓	✓
2	Mark Todo as Done	✓	✓	✓
3	Update Todo Description	✓	✓	✓
4	Filter Todos by Status	✓	✓	✓
5	Delete Todo	✓	✓	✓
6	Create Project	✓	✓	✓
7	Mark Project as Completed	✓	✓	✓
8	Retrieve Active Projects	✓	✓	✓
9	Add Todo to Project	✓	✓	✓
10	Remove Todo from Project	✓	✓	✓
11	Create Category	✓	✓	✓
12	Update Category Description	✓	✓	✓
13	Add Category to Project	✓	✓	✓
14	Retrieve Projects of Category	✓	✓	✓
15	Delete Category	✓	✓	✓
16	Add Todo to Category	✓	✓	✓
17	Remove Todo from Category	✓	✓	✓
18	Retrieve Projects from Todo	✓	✓	✓
19	Delete Project (connected)	✓	✓	✓
20	Add Duplicate Todo to Project	✓	✓	✓

Table 1: List of 20 user stories and their acceptance flows (Normal, Alternate, Error).

2.2 Acceptance test flows

For every user story, we implemented at least three distinct acceptance tests:

- **Normal flow:** Validates the primary path where the system behaves as expected under standard conditions (e.g. successfully creating a TODO, updating a description, or filtering by status).
- **Alternate flow:** Validates secondary paths (e.g. updating both title and description in one operation, or viewing completed rather than incomplete TODOs).
- **Error flow:** Validates robustness (non-existent IDs, invalid payloads or query parameters). Where the API behaviour is known to be incorrect, the scenario is marked as Error Flow / BUG and the test asserts the *expected* correct behaviour (e.g. 400 **Bad Request** for invalid input) so that it fails until the bug is fixed.

As an illustration, Listing 1 shows the Error Flow scenario from Story 3. The scenario sends an invalid (empty) payload to `PUT /todos/:id` and expects the API to return 400 with a validation message; the test currently fails because the API returns 200.

```

Scenario Outline: Update a TODO with invalid data (Error Flow / BUG)
  When the student attempts to update the TODO with title "<title>"
    to have invalid payload
  Then the student is notified of the failed validation with a
    message "Failed Validation"
Examples:
  | title          |
  | Read textbook |

```

Listing 1: Story 3 Error Flow scenario: invalid payload should yield 400 and a validation message.

3 Source Code Repository

The automation framework was developed in JavaScript using Cucumber.js. The repository is hosted on GitHub and is structured to prioritise clean code, reusability, and strict isolation between tests, following the rubric’s requirement for clean, well-structured code.

3.1 Step definition library

Step definitions are organised as a centralised library to avoid duplication and to allow reuse across feature files:

- **Modular API wrapper (api.js):** All HTTP requests (GET, POST, PUT, DELETE) are encapsulated in dedicated helper functions using Axios. This abstracts the network layer and exposes a `request` function for low-level calls when needed (e.g. sending invalid query parameters).
- **Reusable steps (sharedSteps.js):** Common steps such as “the student is notified of the completion of the creation/update/deletion/query operation,” “the student is notified of the failed validation with a message,” and “the student is notified of the non-existence error with a message” are centralised and parameterised so that many feature files share the same step definition.
- **Story-specific steps:** Each story has a corresponding step definition file (e.g. `Story3UpdateTodoDescriptionSteps.js`, `Story4FilterTodosByStatusSteps.js`) that imports `api.js` and `sharedSteps.js` where needed and defines steps unique to that story.

3.2 Key files and directory layout

The story test suite lives under `partB/story_test_suite/`. The `features/` directory contains the 20 Gherkin feature files; the `step_definitions/` directory contains `api.js`, `sharedSteps.js`, `hooks.js`, and one `Story*Steps.js` file per story.

3.3 System state and independence

To satisfy the rubric requirement that story tests can run in any order and restore the initial state upon completion:

- **Hooks (hooks.js):** Cucumber `Before` and `After` hooks run for every scenario. They clean the database and restore a predictable state; resources created during a scenario (e.g. TODOs, projects) are registered and deleted in `After`.
- **Independent execution:** Because state is reset and re-seeded via `Background` for each scenario, there is no data bleed between scenarios. The accompanying video demonstrates that the suite behaves correctly when run with `-order random`.

3.4 Running the tests and server dependency

Before running the tests, the Todo Manager REST API must be started (e.g. `java -jar runTodoManagerRestAPI-1.5.5.jar`) so that it is available at `http://localhost:4567`. From the repository root, install dependencies and run the suite as follows:

```
cd partB/story_test_suite
npm install
npm test    (default order)
npm run test:random  (random order, seed printed in console)
```

A global check verifies that the API is running before any operations. If the server is offline, the suite fails gracefully and immediately rather than hanging or producing misleading assertion errors. Figure 1 shows a typical test run in which the suite is executed in random order and the summary (79 scenarios, 7 failed, 72 passed) is displayed in the console.

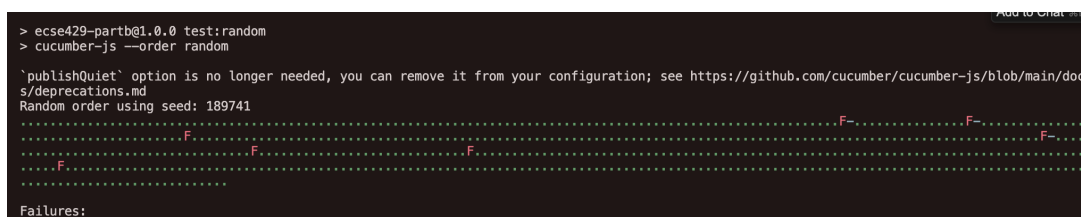


Figure 1: Cucumber test suite execution (random order and scenario summary with 7 fails all described in bug summary).

4 Findings of Story Test Suite Execution

Execution of the story test suite provided confidence in the API's standard operations but revealed several edge-case behaviours and validation gaps. The Todo Manager REST API supports all 20 stories under the interpretations described in Section 1; the bugs listed below were discovered during automated execution and are documented in the accompanying bug summary PDFs.

4.1 Test execution summary

Metric	Value
Total scenarios executed	79
Total steps executed	442
Passed scenarios	72
Failed scenarios	7
Pass rate	72/79 (91.1%)

Table 2: Story test suite execution summary.

The seven failing scenarios correspond to known API defects. The tests are written to assert the *expected* correct behaviour (e.g. 404 for non-existent resources, 400 for invalid input); they therefore fail until the API is fixed and act as regression tests for these bugs.

4.2 Bug summary

The following six bugs were identified. The first four are documented in `partB/bug_summaries/` with the filenames indicated; the last two are documented in `Story3Bug.pdf` and `story4Bug.pdf`.

1. **Non-existing TODO ID returns all projects (Story 18: Retrieve all projects associated with a todo).**
The error flow is in `Story18RetrieveProjectsFromTodo.feature`: “Retrieve all projects for a non-existing todo (Error Flow).” When sending `GET /todos/:id/tasksof` with a non-existing TODO ID (e.g. 99999), the API should return 404 Not Found with an appropriate error message. Instead, it returns 200 OK with the full list of all projects in the API, indicating that the non-existence check is not performed for this endpoint. Bug summary: `Bug_NonExistingTodoReturnsAllProjects.pdf`.
2. **Completed field incorrect when filtering projects (Story 8: Retrieve all active projects).**
The scenario is in `Story8RetrieveActiveProjects.feature`: “Query only genuinely in-progress projects using both filters (Normal Flow).”
When querying `GET /projects?active=true&completed=false`, the API should return only projects where both `active` is true and `completed` is false. Instead, the returned projects include entries where `completed` is true, so the `completed` filter is not correctly applied or the response field is misreported.
Bug summary: `Bug_CompletedFieldMismatch.pdf`.
3. **Bidirectional tasksof relationship (Story 20: Add duplicate todo to project).**
The error flow is in `Story20AddDuplicateTodoToProject.feature`: “Add todo to project via project endpoint and verify the project is not visible from the todo’s tasksof (Error Flow).” When a TODO is added to a project via `POST /projects/:id/tasks`, the relationship should be visible only from the project side. However, the project also appears in `GET /todos/:id/tasksof`, so the API incorrectly creates a bidirectional relationship.
Bug summary: `Bug_BidirectionalTasksofRelationship.pdf`.
4. **Active filter ignored for category projects (Story 14: Retrieve all projects of a specific category).**
The scenario is in `Story14RetrieveProjectsOfCategory.feature`: “Retrieve all active projects assigned to a category (Normal Flow).”
When querying `GET /categories/:id/projects?active=true`, the API should return only projects that are assigned to the category and have `active` set to true. Instead, it returns an empty list even when such projects exist, so the `active` filter is not applied when querying projects through the category endpoint. Bug summary: `Bug_ActiveFilterCategory.pdf`.
5. **Invalid TODO update returns 200 (Story 3: Update a TODO description).**
The error flow is in `Story3UpdateTodoDescription.feature`: “Update a TODO with invalid data (Error Flow / BUG).” When sending `PUT /todos/:id` with an invalid or empty payload, the API should return 400 Bad Request with a validation error message (e.g. “Failed Validation”). Instead, it returns 200 OK. The test asserts the expected 400 and therefore fails until the API is fixed. Bug summary: `Story3Bug.pdf`.
6. **Invalid doneStatus filter returns 200 (Story 4: Filter TODOs by completion status).**
The error flow is in `Story4FilterTodosByStatus.feature`: “Filter TODOs with invalid doneStatus value (Error Flow / BUG).” When querying `GET /todos?doneStatus=maybe` (or another invalid value), the API should return 400 Bad Request with a validation error. Instead, it returns 200 OK. The test asserts the expected 400 and therefore fails until the API is fixed. Bug summary: `story4Bug.pdf`.

5 Conclusion

The implementation of the Cucumber test suite successfully automated the validation of 20 user stories against the Todo Manager REST API. By structuring the automation into a reusable step-definition library and enforcing strict state-reset protocols in `hooks.js`, we ensured a robust, order-independent test environment that satisfies the rubric requirements: story tests run in any order, restore the initial state upon completion, and fail if the system is not running.

The accompanying video demonstration shows the full suite executing in two different orders (including `npm run test:random`) and the fail-safe behaviour when the API service is offline. The discovery of six bugs through Error and Alternate flows highlights the value of BDD in uncovering edge-case and validation defects that manual testing might miss. The seven failing scenarios serve as explicit regression tests until the API is corrected; once the API returns the expected status codes and messages, those scenarios will pass.